

# OS Simulator: Process Scheduling & Virtual Memory Management

## 1. Introduction

The objective of this project is to design and implement a low-level simulator for the core components of an Operating System: the Process Scheduler and the Virtual Memory Manager.

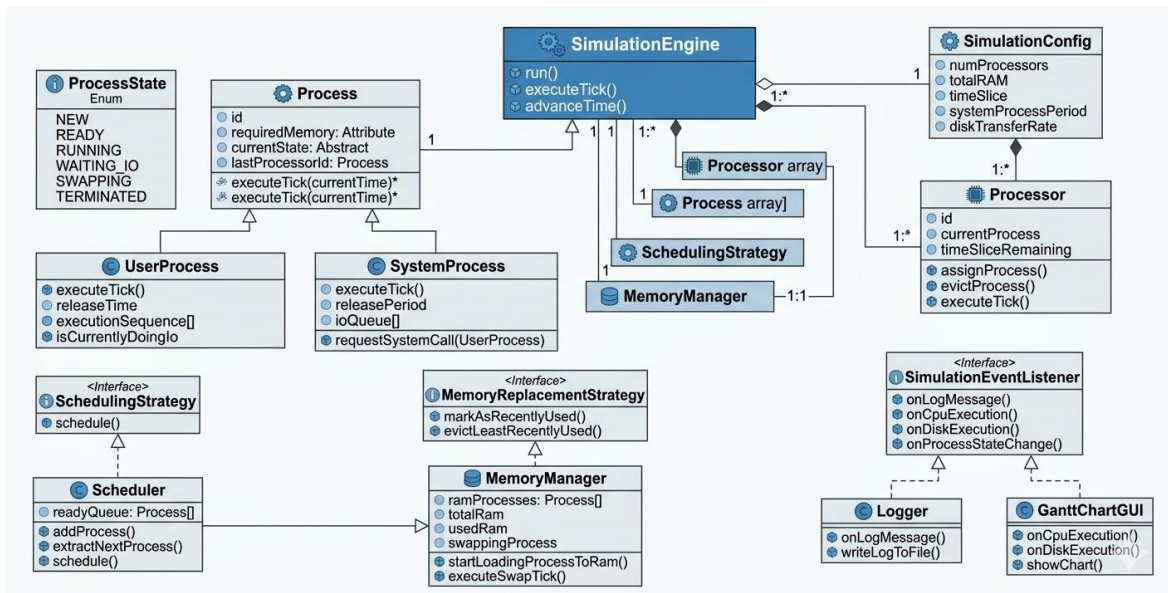
The simulator models a multiprocessor environment where user processes are scheduled using a preemptive Round-Robin algorithm with processor affinity. Additionally, it simulates hardware limitations by implementing a Virtual Memory paging system utilizing the Least Recently Used replacement policy. A dedicated, high-priority System Process is also modeled to handle periodic I/O system calls requested by user processes.

## 2. Program Design and Implementation

### 2.1. System Architecture

The system adopts a highly decoupled, modular architecture utilizing established Design Patterns to ensure maintainability and separation of concerns:

- **Model-View-Controller (MVC) principle:** The core business logic is strictly isolated from input parsing and graphical output.
- **Strategy Pattern:** The interfaces `SchedulingStrategy` and `MemoryReplacementStrategy` (allowing algorithms to be swapped without altering the main engine).
- **Observer Pattern:** The `SimulationEngine` acts as the observable subject. It broadcasts execution events to registered listeners via the `SimulationEventListener` interface, entirely decoupling the simulation logic from the UI.



## 2.2. Process Lifecycle

Throughout the simulation, user processes transit through a strict state machine:

| Process State     | Description                                                                   |
|-------------------|-------------------------------------------------------------------------------|
| <b>NEW</b>        | Waiting for the global clock to reach its release time.                       |
| <b>SWAPPING</b>   | Scheduled for execution, but awaiting data transfer.                          |
| <b>READY</b>      | Loaded in RAM. Waiting in the scheduler for a free CPU.                       |
| <b>RUNNING</b>    | Assigned to a CPU. Actively consuming its execution burst.                    |
| <b>WAITING_IO</b> | Blocked on a system call. Waiting in the System Process queue for resolution. |
| <b>TERMINATED</b> | Execution sequence fully completed. Evicted from the system permanently.      |

## 2.3. Core Components and Implementation Details

The simulation is driven by several modules:

### A. The Simulation Engine

This component acts as the global system clock. It runs a continuous loop, advancing the global time and orchestrating the system sequence:

1. Validates and introduces new processes into the system based on their release time.
2. Triggers a disk to RAM transfer tick if virtual memory is actively swapping.
3. Wakes up the System Process if its periodic timer has expired.
4. Advances the execution state of processes currently loaded on processors.
5. Delegates processor reassignment to the Scheduler.

## B. The Scheduler

Implements the preemptive Round-Robin algorithm.

- **Data Structure:** Utilizes a circular queue to manage processes in the ready state.
- **Affinity Logic:** When allocating available processors to user processes, the system prioritizes scheduling a process on the CPU it most recently executed on, thereby respecting the affinity rule and minimizing context-switching overhead.
- **Priority:** The System Process bypasses the standard queue. If it is ready, it instantly preempts available CPU resources.

## C. Virtual Memory Manager

Simulates physical RAM limitations and Disk transfers.

- **LRU Policy:** Managed via a native array. When a process executes a CPU tick, `markAsRecentlyUsed()` shifts it to the end of the array.
- **Swapping:** The time required to load a process is calculated mathematically ( $\text{RequiredMemory} / \text{DiskTransferRate}$ ). While a transfer is active, the requested process remains in the swapping state.

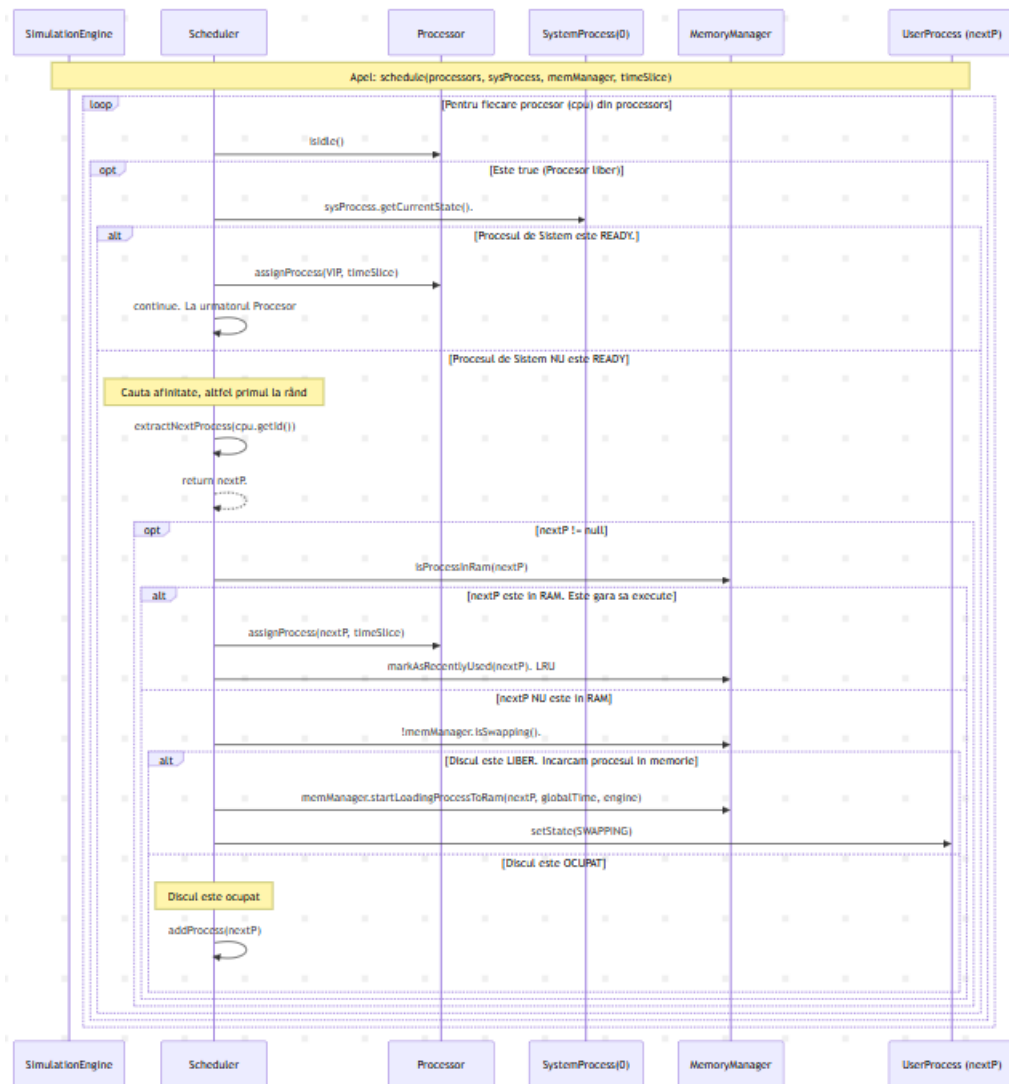
## D. The System Process

A high-priority entity that executes system calls (I/O). It has a memory footprint of 0 and resides permanently in RAM. It maintains its own manual circular buffer to queue user processes that have blocked for I/O. When its queue is empty, it transitions to a sleep state, releasing the CPU.

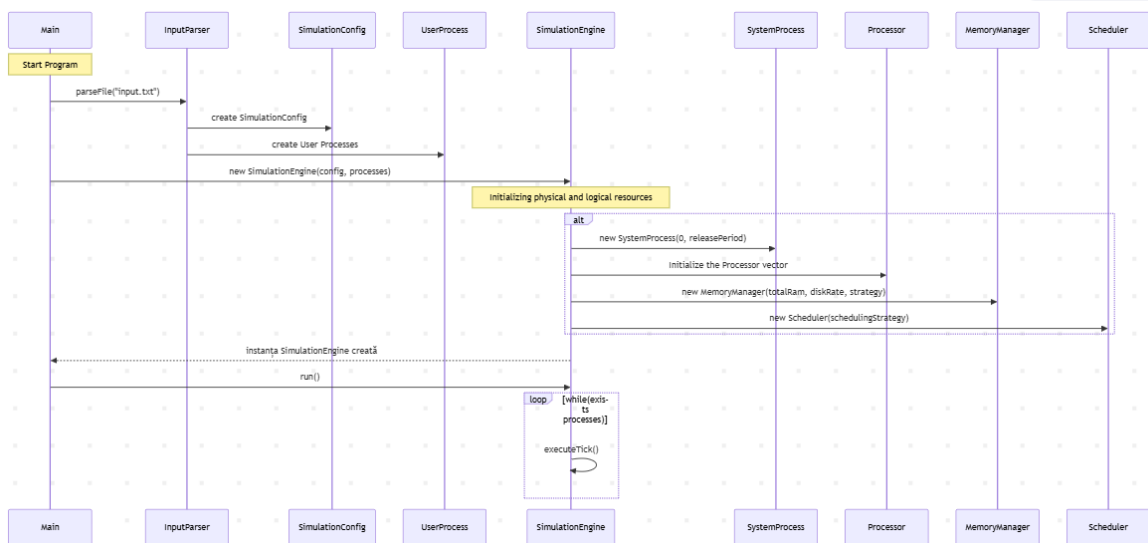
## 2.4. Sequence Diagrams

The dynamic execution flow and module interactions are illustrated through two critical sequence diagrams:

- **Scheduler Execution Flow:** Maps the micro-step logic executed during each clock tick. It outlines the CPU allocation process: handling high-priority System Process I/O tasks, applying affinity rules for user processes in the ready queue, and triggering disk to RAM swapping if a scheduled process is missing from physical memory.



- **Application Initialization Flow:** Tracks the complete startup sequence. It illustrates how the system parses the input file, initializes hardware components, binds strategy implementations, and registers output listeners before starting the clock.



### 3. User Manual

#### 3.1. Prerequisites and Execution

- **Environment:** Java Development Kit (JDK) 8 or higher.
- **Setup:** Ensure the input configuration file *input.txt* is placed in the root execution directory.
- **Execution:** Run the Main.java class. The system operates autonomously upon launch, strictly based on the configuration and process structure provided in the input.txt file.

#### 3.2. Input File Format

The simulator requires a strictly formatted text file consisting of the system configuration and process definitions (one line per process).

| Line                 | Parameters (Space-separated)                                              |
|----------------------|---------------------------------------------------------------------------|
| <b>System Config</b> | [Processors] [Total RAM] [Time Slice] [System Process Period] [Disk Rate] |
| <b>Process</b>       | [Release Time] [RAM Required] [CPU_Burst] [IO_Burst] [CPU_Burst]          |

*Example:*

System Configuration:      2 1024 5 15 10  
Process:                      0 256 10 2 5 ...

#### 3.3. Output and Interpretation

The simulator provides output via three channels:

1. **Console Logging:** Real-time text traces of system events (evictions, affinity matches, swap completions).
2. **File Logging (output.txt):** A persistent copy of the console trace for historical analysis.
3. **Graphical Interface:** A Java Swing window visualizing the timeline. CPUs and the Hard Disk are separated by rows. Processes are color-coded, with the System Process marked distinctly.

### 4. Unit Testing and Component Isolation

#### 4.1. Objectives

The goal of the testing phase was to ensure the robustness, correctness, and fault-tolerance of the components.

The testing relies heavily on Unit Testing combined with an Incremental Bottom-Up integration strategy. The focus was not just on verifying the "happy path", but also on ensuring that the system gracefully handles incorrect input data, capacity overflows, and edge cases without fatal crashes.

| Element ^                    | Class, %     | Method, %    | Line, %        | Branch, %     |
|------------------------------|--------------|--------------|----------------|---------------|
| ✓ all                        | 100% (14/14) | 100% (78/78) | 100% (470/470) | 98% (213/216) |
| └─ simulator.config          | 100% (1/1)   | 100% (7/7)   | 100% (12/12)   | 100% (0/0)    |
| └─ SimulationConfig          | 100% (1/1)   | 100% (7/7)   | 100% (12/12)   | 100% (0/0)    |
| └─ simulator.core            | 100% (3/3)   | 100% (23/23) | 100% (195/195) | 98% (110/112) |
| └─ MemoryManager             | 100% (1/1)   | 100% (9/9)   | 100% (53/53)   | 100% (28/28)  |
| └─ MemoryReplacementStrategy | 100% (0/0)   | 100% (0/0)   | 100% (0/0)     | 100% (0/0)    |
| └─ Scheduler                 | 100% (1/1)   | 100% (6/6)   | 100% (63/63)   | 100% (30/30)  |
| └─ SchedulingStrategy        | 100% (0/0)   | 100% (0/0)   | 100% (0/0)     | 100% (0/0)    |
| └─ SimulationEngine          | 100% (1/1)   | 100% (8/8)   | 100% (79/79)   | 96% (52/54)   |
| └─ simulator.io              | 100% (5/5)   | 100% (20/20) | 100% (170/170) | 98% (67/68)   |
| └─ GanttChartGUI             | 100% (3/3)   | 100% (11/11) | 100% (104/104) | 100% (30/30)  |
| └─ InputParser               | 100% (1/1)   | 100% (4/4)   | 100% (53/53)   | 97% (33/34)   |
| └─ Logger                    | 100% (1/1)   | 100% (5/5)   | 100% (13/13)   | 100% (4/4)    |
| └─ SimulationEventListener   | 100% (0/0)   | 100% (0/0)   | 100% (0/0)     | 100% (0/0)    |
| └─ simulator.model           | 100% (5/5)   | 100% (28/28) | 100% (93/93)   | 100% (36/36)  |
| └─ Process                   | 100% (1/1)   | 100% (7/7)   | 100% (11/11)   | 100% (0/0)    |
| └─ Processor                 | 100% (1/1)   | 100% (8/8)   | 100% (21/21)   | 100% (8/8)    |
| └─ ProcessState              | 100% (1/1)   | 100% (2/2)   | 100% (7/7)     | 100% (0/0)    |
| └─ SystemProcess             | 100% (1/1)   | 100% (6/6)   | 100% (32/32)   | 100% (12/12)  |
| └─ UserProcess               | 100% (1/1)   | 100% (5/5)   | 100% (22/22)   | 100% (16/16)  |

## 4.2. Testing Strategy

### 4.2.1. Bottom-Up Incremental Testing

The testing process was designed following a strict Bottom-Up hierarchy. We first tested the independent foundational modules (configurations, states). Once verified, these modules were trusted and used to test higher-level execution entities (processes, processors), eventually culminating in the testing of the highly complex algorithmic modules (Scheduler, Memory Manager) and the central SimulationEngine.

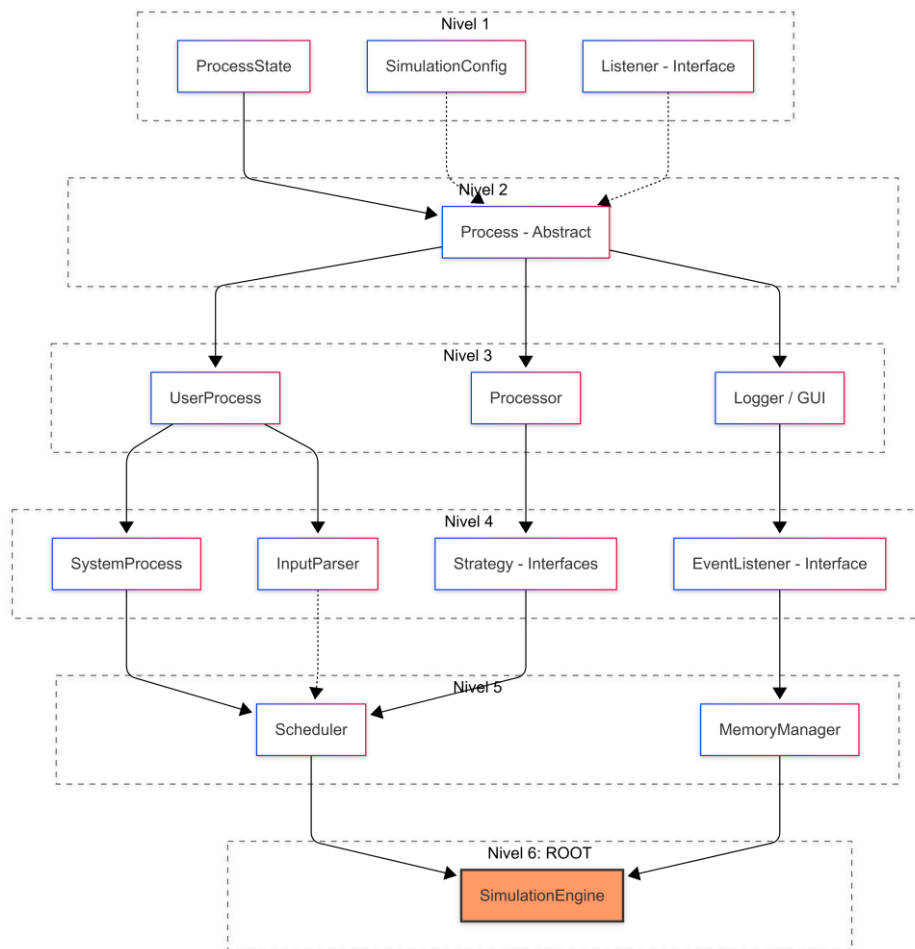
### 4.2.2 Module Isolation & Mocking

To ensure that each module is tested independently, Mockito was used extensively. By injecting mock objects (e.g., mocking the MemoryManager when testing the Scheduler), we isolated the logic of the specific class under test. This prevents failures in lower-level components from cascading and causing false negatives in higher-level tests.

### 4.2.3. Dependency Architecture

The following diagram illustrates the Bottom-Up dependency tree used to dictate the order of test execution.

The diagram illustrates how Level 1 components are tested first, moving up layer by layer until reaching the Level 6 SimulationEngine.



### 4.3. Detailed Module Test Breakdown

#### Level 1 & 2: Foundations & Models

- SimulationConfigTest (100% Coverage)
  - What was tested: Verifies the correct instantiation of immutable system parameters (RAM, CPU count, Time Slice). Tests the toString() method for correct formatting.
- ProcessState & SimulationEventListener
  - Enums and interfaces inherently hold no logic and act as contracts, requiring no explicit unit tests.

#### Level 3: Execution Entities

- ProcessorTest (100% Coverage)
  - What was tested: CPU initialization, assigning processes (affinity tracking), and eviction logic. Evaluates the executeTick() method to ensure the time slice decrements correctly.
  - Fault Tolerance: Tests assigning a null process to ensure the CPU handles empty states without throwing NullPointerExceptions.

- UserProcessTest (100% Coverage)
  - What was tested: Alternating states between CPU bursts and I/O blocking.
  - Incorrect Input Handling: Tests sequences that are null, empty, or contain negative burst values. The logic successfully transitions the process to a TERMINATED state or switches to I/O immediately, preventing infinite loops.

#### Level 4: Services & Input/Output

- SystemProcessTest (100% Coverage)
  - What was tested: The system process periodic wakeup trigger (checkReleaseTime), servicing I/O bursts for user processes, and transitioning to WAITING\_IO when idle.
  - Boundary Limits: Tests pushing over 1,000 items into the manual circular queue to ensure capacity limits are respected without crashing.
- InputParserTest (~97% Branch Coverage)
  - What was tested: Manual, character-by-character string parsing (bypassing standard library split functions).
  - Incorrect Input Handling: Tests irregular spacing, tabs, and non-numeric characters. It also tests providing a non-existent file path, ensuring the IOException is caught and handled safely internally.
  - *Coverage Note*: The missing 3% on branches corresponds to deep underlying I/O stream exceptions that are heavily dependent on the OS file system and are difficult to safely trigger in a standard unit test.
- LoggerTest (100% Coverage)
  - What was tested: File creation, message writing, and stream closure. Includes tests for missing/invalid file paths to ensure the fallback protection (if writer != null) works correctly.

#### Level 5: Core Algorithms

- SchedulerTest (100% Branch Coverage)
  - What was tested: The Round-Robin scheduling logic, process queue management, and the disk swap-in trigger. Tests the VIP priority system (ensuring SystemProcess takes the CPU immediately when ready).
  - Affinity Logic: Tests the queue iteration to verify that the scheduler successfully searches for and extracts a process that previously ran on the available CPU.
  - Isolation: MemoryManager and SimulationEngine were fully mocked to test CPU assignment purely based on state.
- MemoryManagerTest (100% Coverage)
  - What was tested: The manual Least Recently Used (LRU) array shifting logic, RAM capacity calculations, and Swapping tick math (Math.ceil behavior).

- Edge Cases: Evicting from an empty RAM, marking a process as MRU when it is already the most recently used, and mathematical edge cases where a process requires 0 memory.

## Level 6: Integration

- SimulationEngineTest (~96% Branch Coverage)
  - What was tested: The central clock tick orchestrator. Uses Java Reflection to inject mocked subsystems (Scheduler, CPU, Memory). Tests the interaction flow: Process Launch -> CPU Assign -> Preemption -> RAM Swapping -> I/O Blocking -> VIP Servicing.
  - Coverage Note: The slightly less than 100% branch coverage is expected here. The engine contains highly defensive else statements guarding against impossible concurrent states (e.g., a process terminating exactly while being swapped in under a specific edge-case tick).
- GanttChartGUITest.java (UI Coverage)
  - What was tested: Tracking blocks of execution for CPUs and the Hard Disk. Simulates a Graphics2D context to trigger the paintComponent method.
  - Coverage Note: UI rendering classes (Java Swing) generally do not reach 100% branch coverage in unit tests because native OS rendering events, font metrics, and window manager callbacks cannot be entirely replicated in a headless testing environment. The logic for block coordinate tracking, however, was fully verified.

## 5. Design by Contract (Assertions)

In parallel with Unit Testing, Design by Contract (DbC) was enforced directly within the application's source code. This phase utilizes the native Java assert keyword to mathematically and logically prove the correctness of the code during runtime.

These assertions act as strict guards, instantly halting the JVM with an AssertionError if a component violates its contract, ensuring errors fail fast and close to their source.

### 5.1. Preconditions (Input and State Validation)

Preconditions define the absolute requirements that must be true *before* a method executes. They protect the module from invalid inputs provided by external callers or incorrect internal states.

- Input Validation: Protects against null references and invalid mathematical parameters.

- *Example (Processor)*: assert process != null : "Cannot assign a null process to CPU"
- *Example (Configuration)*: assert timeSlice > 0 : "Time slice quantum must be greater than 0"
- State Validation: Ensures the system is in the correct lifecycle phase to perform the requested action.
  - *Example (MemoryManager)*: assert !isSwapping() : "Cannot start a new swap-in; disk is currently busy"
  - *Example (Processor)*: assert !isIdle() : "Cannot evict from an idle processor"
  - *Example (Scheduler)*: assert size < capacity : "CRITICAL ERROR: Scheduler Ready Queue is full!"

## 5.2. Postconditions (Execution Verification)

Postconditions act as strict guarantees regarding the outcome of a method. They mathematically prove that the internal state transition occurred exactly as intended, comparing the before and after states.

- Mathematical Progression: Verifies that timers and counters advance monotonically.
  - *Example (SimulationEngine)*: assert globalTime == previousTime + 1 : "Simulation clock failed to step forward monotonically"
  - *Example (Processor)*: assert timeSliceRemaining == previousSlice - 1 : "Time slice must decrement by exactly 1"
- State Transition Guarantees: Proves that an object successfully transitioned to its new expected state.
  - *Example (Scheduler)*: assert p.getCurrentState() == ProcessState.READY : "Enqueued process was not set to READY state"
- Capacity and Metric Tracking: Ensures arrays and counters track data accurately after manipulations.
  - *Example (SystemProcess)*: assert queueSize == previousSize - 1 : "Queue size must decrease by 1 upon completion"

## 5.3. Class Invariants (Structural Integrity)

Because the simulator is restricted from using standard Java Collections (like LinkedList or ArrayList), complex data structures such as circular buffers and LRU tracking arrays were implemented manually. Class invariants are overarching rules that must hold true at all times (except momentarily during the execution of a private mutation).

To enforce this, dedicated private helper methods were implemented and asserted globally before and after any public operation:

- **The Circular Queue Invariant (`checkQueueInvariant()`):** Used in both Scheduler and SystemProcess, this method mathematically proves that the manual array pointers (head and tail) never breach the array boundaries and that the current size never exceeds capacity or drops below zero.
  - *Execution:* Evaluated before adding/extracting a process, and immediately after the pointer math  $((\text{tail} + 1) \% \text{capacity})$  resolves.
- **The Virtual Memory Invariant (`checkMemoryInvariant()`):** Used in the MemoryManager to guarantee physical RAM boundaries are respected during LRU eviction and Disk Swapping.
  - *Rule 1:* Proves that the tracked usedRam never exceeds totalRam and never drops below 0.
  - *Rule 2:* Proves that the swap timer (swapTicksRemaining) never becomes negative.
  - *Execution:* Validated before initiating a Swap-In, after an LRU eviction, and at the end of every executeSwapTick().

#### 5.4. The Fail-Fast Philosophy

By embedding these assertions deeply into the architecture, the OS Simulator effectively documents its own logic constraints natively in the code. If a future modification accidentally breaks the Round-Robin pointer math or over-allocates the RAM, the system will immediately halt with a highly specific error message, rather than allowing the simulator to generate corrupted, unpredictable Gantt charts.

## 6. Team Contributions

### 6.1 System Design & Core Implementation

All three team members contributed equally to the architectural design, the definition of the process state machine, and the implementation of the core execution logic. The strict constraint of avoiding standard libraries was tackled jointly through pair programming to ensure uniform code quality across all files.

### 6.2 Testing & Assertions

Once the foundational code was stable, the testing and assertion phases were divided by module to ensure deep, specialized coverage:

- Popoiu Andra Stefania – Virtual Memory Management:
  - *Testing:* Developed the hybrid integration test suite for the MemoryManager. Verified the mathematical edge cases of the LRU eviction policy, continuous swapping resolution, and zero-memory processes (MemoryManagerTest).

- Assertions: Implemented and enforced the `checkMemoryInvariant()` to mathematically guarantee that physical RAM boundaries are strictly respected during evictions and load operations.
- Stoinea Miruna Maria – Process Scheduling:
  - Testing: Wrote the test suite for the Scheduler. Used Mockito to verify processor affinity routing, VIP priority overrides, and the safety limits of the manual circular queue (`SchedulerTest`).
  - Assertions: Implemented the `checkQueueInvariant()` to prove the integrity of the manual circular buffer pointers (head, tail, size) and enforced state-transition postconditions for CPU assignments.
- Ungureanu Ana Maria – Simulation Engine & Orchestration:
  - Testing: Authored the central engine test suite (`SimulationEngineTest`). Utilized Java Reflection and Mockito to inject mocked subsystems, verifying the global clock orchestration, event listener fail-safes, and safety timeout mechanisms.
  - Assertions: Enforced global preconditions for system parameters and postconditions ensuring the monotonic progression of the simulation clock (`globalTime`) and process termination counters.